```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from sklearn.preprocessing import MinMaxScaler
```

```
df=pd.read_csv("/content/airplane_passenger_prediction.csv")
```

```
df
```

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121
...	...	...
139	1960-08	606
140	1960-09	508
141	1960-10	461
142	1960-11	390
143	1960-12	432

144 rows × 2 columns

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df["Month"]=pd.to_datetime(df["Month"])
df.set_index("Month",inplace=True)
```

```
df
```

	Passengers
Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121
...	...
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

144 rows × 1 columns

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
scaler=MinMaxScaler(feature_range=(0,1))
```

```
data_scaled=scaler.fit_transform(df)
```

```
def create_sequence(data,seq_length):
  x,y=[],[]
  for i in range(len(data)-seq_length):
    x.append(data[i:i+seq_length])
```

```
    y.append(data[i+seq_length])
  return np.array(x),np.array(y)
```

```
seq_length=5
X,y=create_sequence(data_scaled,seq_length)
```

```
#split data into train-test split
size=int(0.8*len(X))
X_train,X_test=X[:size],X[size:]
y_train,y_test=y[:size],y[size:]
```

```
#Define the LSTM model
model=Sequential([LSTM(50,return_sequences=True,input_shape=(seq_length,1)),
                  LSTM(50),Dense(1)])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argum
  super().__init__(**kwargs)
```

```
model.compile(optimizer='adam',loss='mse')
model.fit(X_train,y_train,epochs=50,batch_size=16,verbose=1)
```

Show hidden output

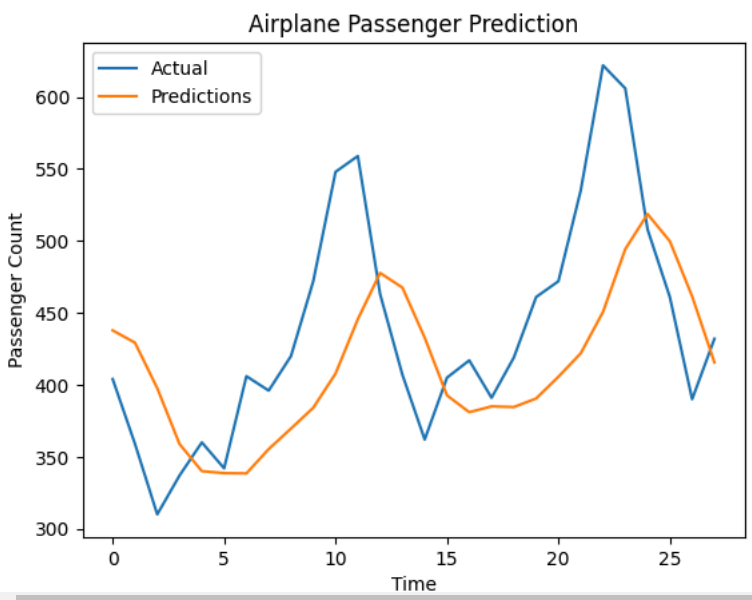
```
loss=model.evaluate(X_test,y_test)
print("Test Loss:", loss)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - loss: 0.0195
Test Loss: 0.0194588303565979
```

```
# Make predictions
predictions = model.predict(X_test)
# Inverse transform to get actual values
predictions = scaler.inverse_transform(predictions)
actual_values = scaler.inverse_transform(y_test)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
```

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(actual_values, predictions)
print("Mean Squared Error:", mse)
```

```
Mean Squared Error: 5221.270443845108
```

```
# Plot the results
plt.plot(actual_values, label="Actual")
plt.plot(predictions, label="Predictions")
plt.legend()
plt.title("Airplane Passenger Prediction")
plt.xlabel("Time")
plt.ylabel("Passenger Count")
plt.show()
```